

Chapter Tow : Lexical Analysis

- Functions of Lexical Analysis
- Role of the Lexical Analyzer
- Tokens, Patterns, and Lexemes
- Specification and Recognition of tokens
- Input Buffering

Chapter Tow : Lexical Analysis

What is Functions of Lexical
Analysis?

Chapter Tow : Lexical Analysis

- Lexical analysis is the first phase of a compiler.
 - The role of the lexical analyzer is to read a sequence of characters from the source program and produce tokens to be used by the parser.
 - The lexical analyzer breaks these sentence (source code) into a series of tokens, by removing any whitespace and comments in the source code.
-

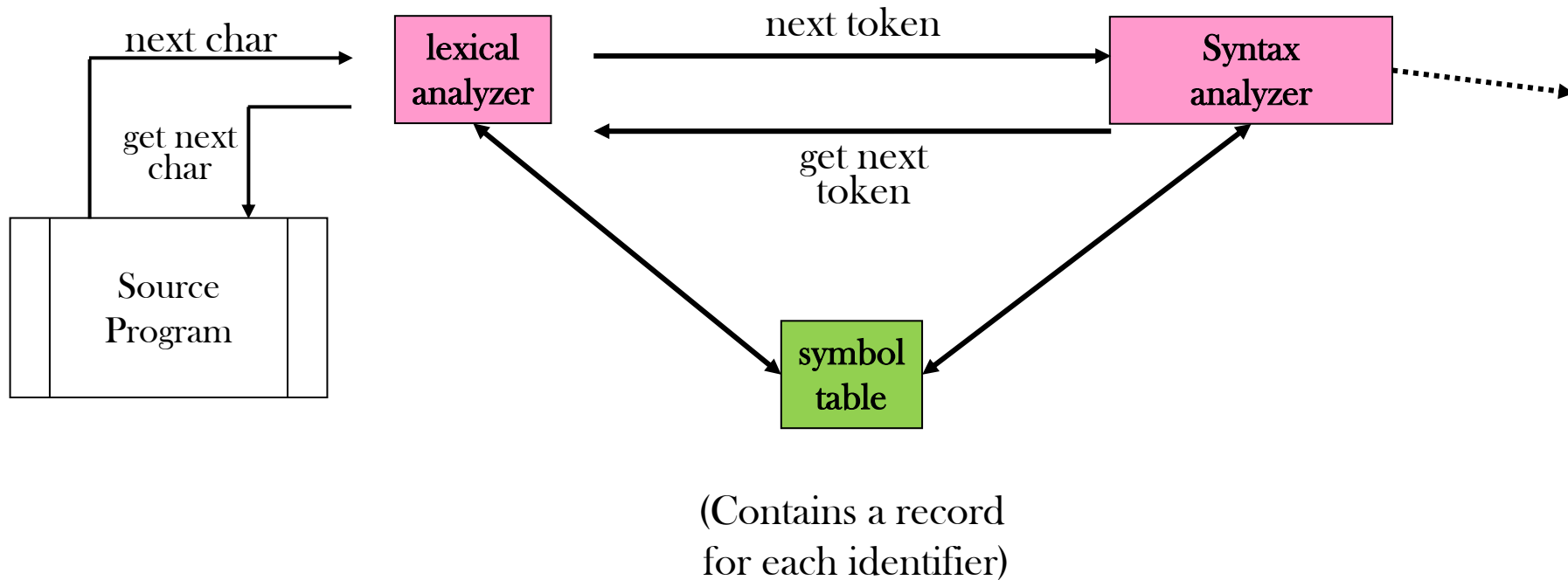
Chapter Tow : Lexical Analysis

- The main task of the lexical analyzer is to read the input characters of the source program, group them into **lexemes**, and produce as an output a sequence of **tokens** for each lexeme in the source program.
 - The stream of tokens is sent to the parser for syntax analysis.
-

Chapter Tow : Lexical Analysis

- The lexical analyzer also interacts with the symbol table, e.g., when the lexical analyzer discovers a **lexeme constituting an identifier**, it needs to enter that lexeme into the symbol table.

Interaction of the Lexical Analyzer with the Parser



token: smallest meaningful sequence of characters of interest in source program

Chapter Tow : Lexical Analysis

What is Role of the Lexical
Analyzer?

Chapter Tow : Lexical Analysis

- The following are additional tasks performed by the lexical analyzer other than identifying lexemes:
 - Stripping out comments and whitespace (blank, newline, and tab) from the source code
 - Correlating error messages generated by the compiler with the source program by keeping track of line numbers.
 - Expanding macros in some lexical analyzers
 - It helps to identify the tokens.
 - The input characters are read by the lexical analyzer from the source code.
-

Chapter Two : Lexical Analysis

Tokens, Patterns, and Lexemes

Token, pattern, lexeme

- A **token** is a sequence of characters from the source program having a collective meaning.
- A **token** is a classification of **lexical units**.
 - For example: **id** and **num**
- **Lexemes** are the specific character strings that make up a **token**.
 - For example: **abc** and **123A**
- **Patterns** are rules describing the set of **lexemes** belonging to a **token**.
 - For example: “*letter followed by letters and digits*”
- Patterns are usually specified using regular expressions.

[a-zA-Z]*

Example: `printf("Total = %d\n", score);`

Token, pattern, lexeme...

- Example: The following table shows some **tokens** and their **lexemes** in java

Token	Some lexemes	pattern
while	While, while, WhiLe...	Begin in small or capital letters
if	If, IF, iF, If	If in small or capital letters
ident	distance, f1, x, dist1,...	Letters followed by zero or more letters and/or digits

- In general, in programming languages, the following are tokens:
 - keywords, operators, identifiers, constants, literals, punctuation symbols...

Chapter Two : Lexical Analysis

Tokens, Patterns, and Lexemes

○ **A token:**

- Is a pair consisting of a token name and an optional attribute value.
 - The token name is an abstract symbol representing a kind of **lexical unit**, e.g., a keyword, an identifier, etc.
 - The token names are input symbols that the parser processes.
 - In any programming language (*Keywords, operators, identifiers, constants, literals, punctuation symbols*) are token
-

Chapter Tow : Lexical Analysis

Tokens, Patterns, and Lexemes

○ **Lexemes:**

- are said to be a sequence of characters (alphanumeric) in a token.
- There are some predefined rules for every lexeme to be identified as a valid token.
- These rules are **defined by grammar rules**, by means of a pattern.
- A **lexeme** is a sequence of characters in the source program that matches the pattern for a token and is identified by the lexical analyzer as an instance of that token

Chapter Tow : Lexical Analysis

Tokens, Patterns, and Lexemes

- A **pattern** is a description of the form that lexemes of a token may take. In case of a keyword as a token, the pattern is the sequence of characters that forms the keyword. For identifiers and some other tokens, the pattern is a more complex structure that is matched by many strings.
- In programming language, keywords, constants, identifiers, strings, numbers, operators and punctuations symbols can be considered as tokens.

For example, in C language, the variable declaration line

```
int value = 100;
```

contains the tokens:

```
int (keyword), value (identifier), = (operator), 100 (constant) and ; (symbol).
```

Chapter Tow : Lexical Analysis

Tokens, Patterns, and Lexemes

Token	Some lexemes	Informal pattern
begin	begin, Begin, BEGIN, beGin, ...	Begin in small or capital letters
if	if, IF, iF, If	if in small or capital letters
ident	Distance, F1, x, Dist1, ...	Letter followed by zero or more letters and/or digits

Chapter Tow : Lexical Analysis

Attributes of tokens

- When more than one pattern matches a lexeme, the scanner must provide additional information about the particular lexeme to the subsequent phases of the compiler.
- For ex., both **0** and **1** match the pattern for the token **num**.
But the code generator needs to know which number is recognized.

Chapter Tow : Lexical Analysis

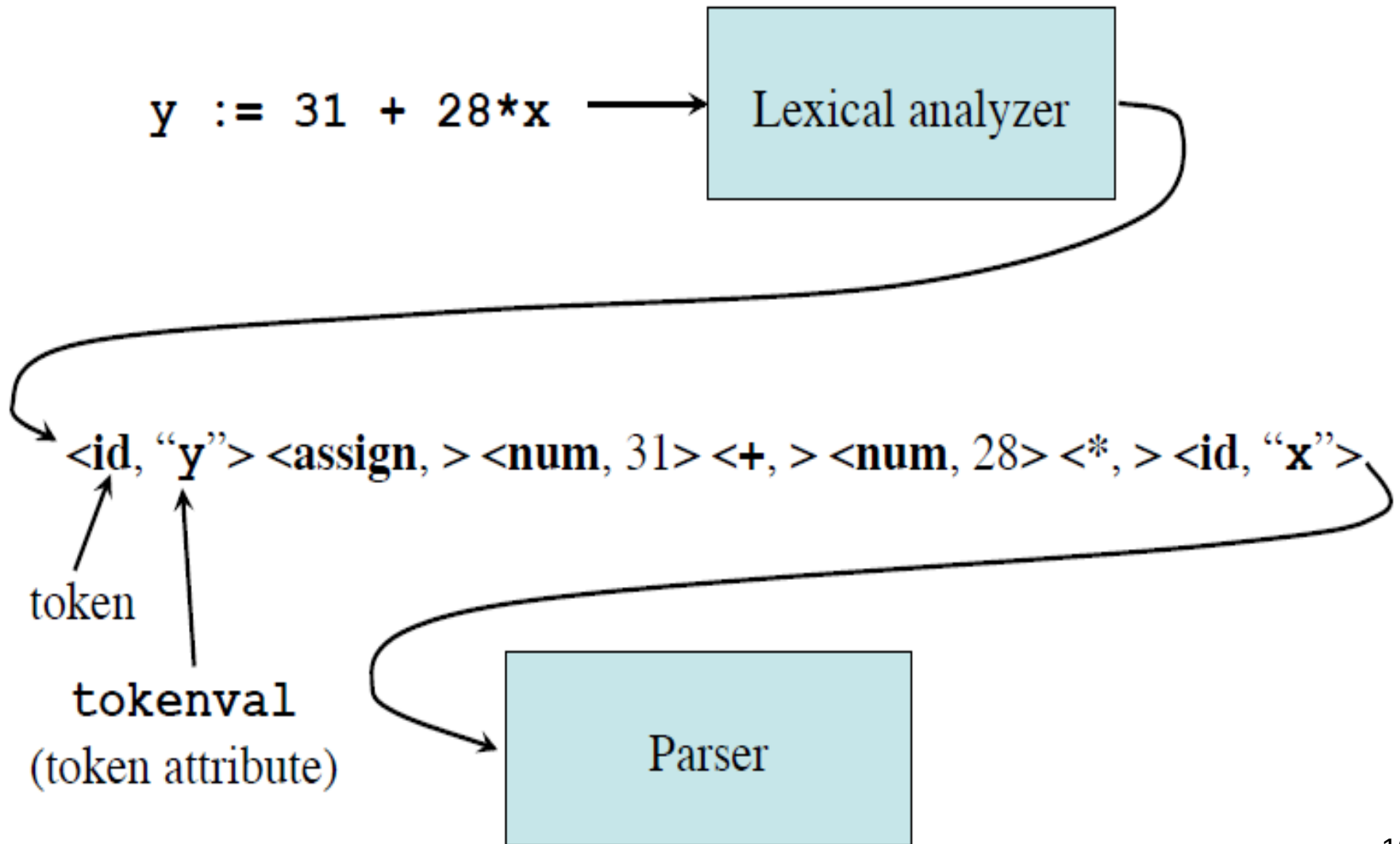
Attributes of tokens

- The lexical analyzer collects information about tokens into their associated attributes.
- Practically, a token has one attribute: a pointer to the symbol table entry in which the information about the token is kept.
- Symbol table entry contains information about the token such as the lexeme, the line number in which it was first seen, ...

Attributes of tokens...

- Practically, a token has one attribute:
 - A pointer to the symbol table entry in which information about the token is kept.
- The symbol table entry contains various information about the token such as its lexeme, type, the line number in which it was first seen ...
 - Ex. $y = 31 + 28 * x$, The tokens and their attributes are written as:

Attributes of tokens...



Chapter Tow : Lexical Analysis

Attributes of tokens

- For ex. consider $x = y + 2$

The tokens and their attributes are written as:

<id, pointer to symbol-table entry for x>

<assign_op, >

<id, pointer to symbol-table entry for y>

<plus_op, >

<num, integer value 2>

Chapter Tow : Lexical Analysis

Errors

- Very few errors are detected by the lexical analyzer.
- For ex., if the programmer mistakes **wihle** for **while**, the lexical analyzer cannot detect the error (why?)
- Nonetheless, if a certain sequence of characters follows none of the specified patterns, the lexical analyzer can detect the error.

Chapter Tow : Lexical Analysis

Errors

- When an error occurs, the lexical analyzer recovers by:
 - skipping (deleting) successive characters from the remaining input until the lexical analyzer can find a well-formed token (panic mode recovery).
 - **deleting** extraneous(unimportant) characters
 - **inserting** missing characters
 - **replacing** an incorrect character by a correct character
 - **transposing** two adjacent characters
-

Chapter Tow : Lexical Analysis

Specifying and Recognizing Tokens

Chapter Tow : Lexical Analysis

Specifying Tokens

- Regular expressions are used to specify the patters of tokens.
- Each pattern matches a set of strings.

Example

letter $\rightarrow$ A|B|C|...|Z|a|b|c|...|z

digit $\rightarrow$ 0|1|...|9

identifier $\rightarrow$ letter (letter|digit)*

Chapter Tow : Lexical Analysis

Recognizing tokens

- Using Transition Diagrams
- As an intermediate step in the construction of a lexical analyzer, we first convert patterns into stylized flowcharts, called “**transition diagrams**”
- Transition diagrams have a collection of nodes **called states**
- Each state represents a condition that could occur during the process of scanning the input looking for a lexeme that matches one of several patterns
- Edges are directed from one state of the transition diagram to another
- Each edge is labeled by a **symbol or a set of symbols**

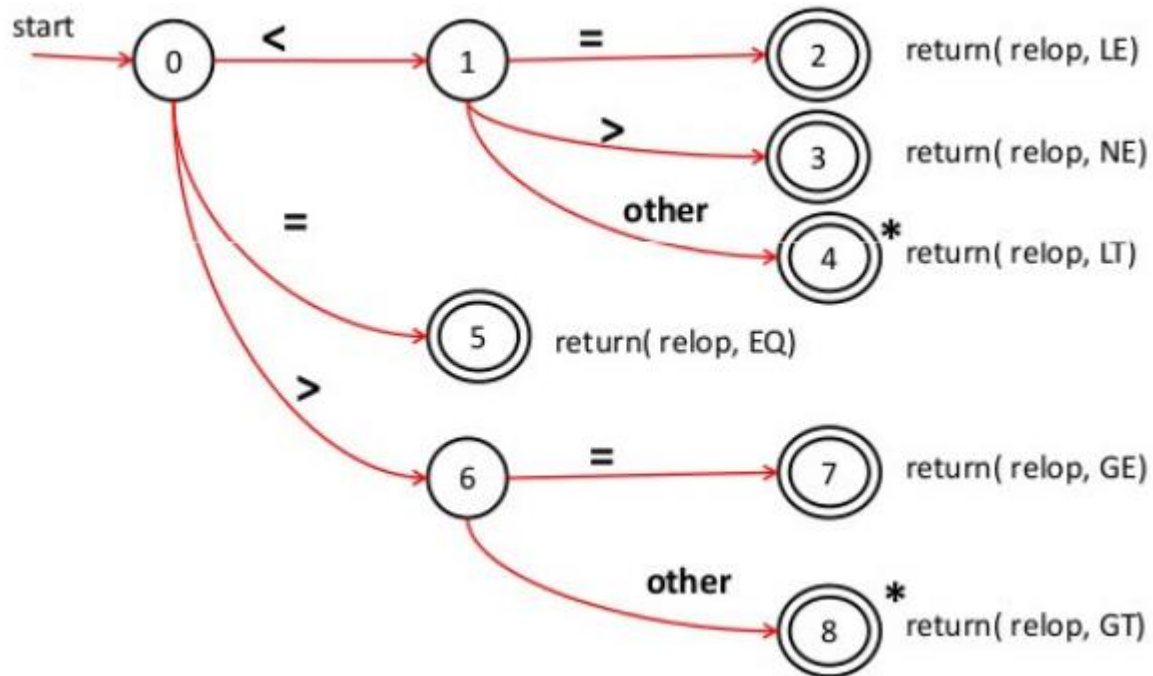
Chapter Tow : Lexical Analysis

Recognizing tokens

- The following figure is a transition diagram that recognizes the lexemes matching the token relational operators in Pascal
- Being on state 0 (initial state), if we see < as the first input symbol, then the lexemes that match the pattern for relop we can only be looking at <, <>, or <=

Chapter Tow : Lexical Analysis

Recognizing tokens



Chapter Tow : Lexical Analysis

Input Buffering

Chapter Tow : Lexical Analysis

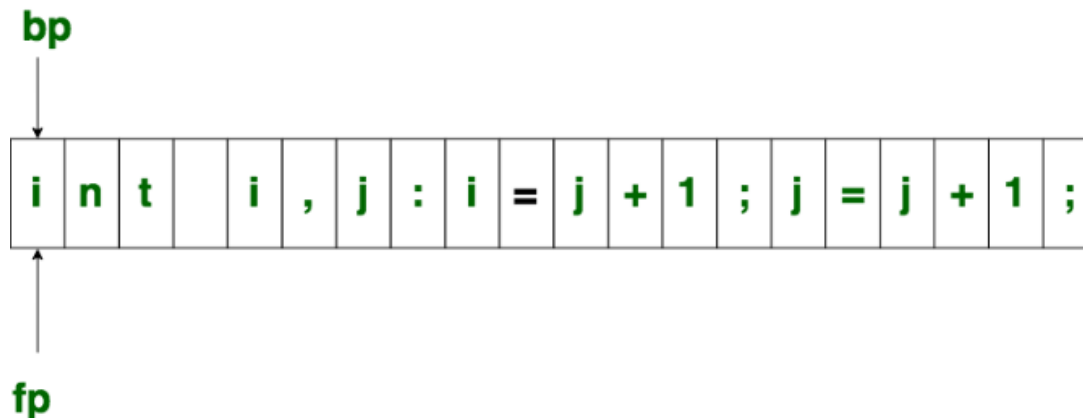
Buffer

- A buffer contains data that is stored for short amount of time, typically in the computer's memory.
- The purpose of a buffer is to hold data right before it is used.

Chapter Tow : Lexical Analysis

Input Buffering

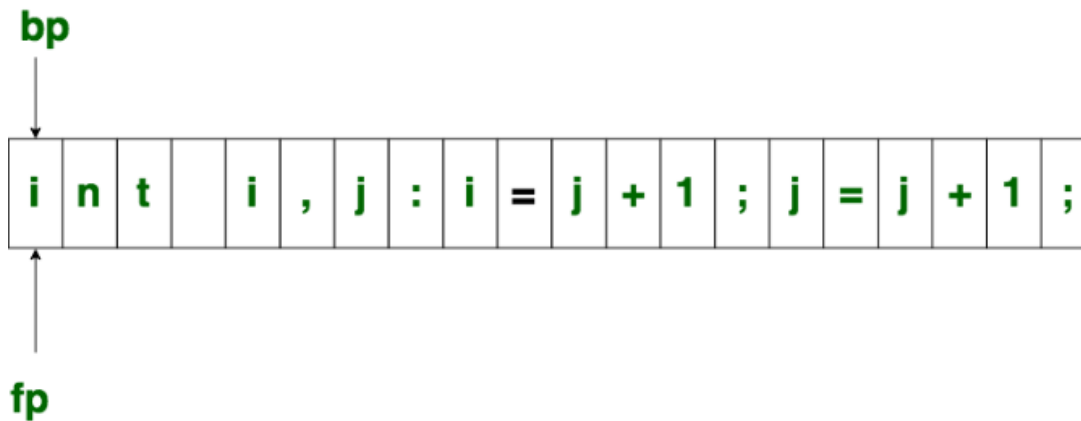
- The lexical analyzer scans the input from left to right one character at a time.
- It uses two pointers begin **ptr(bp)** and forward **ptr(fp)** to keep track of the pointer of the input scanned.



Chapter Tow : Lexical Analysis

Input Buffering

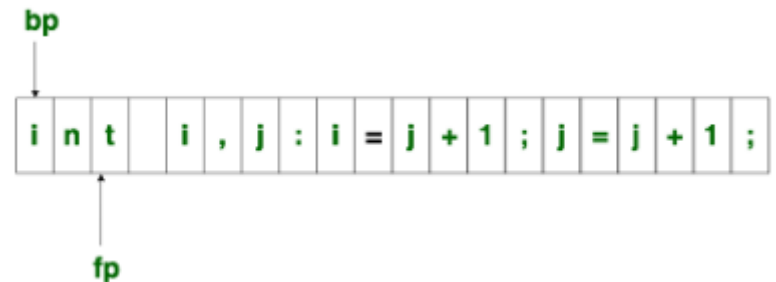
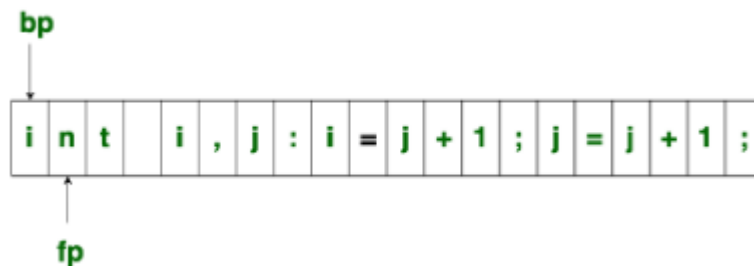
- Initially both the pointers point to the first character of the input string as shown in the figure below



Chapter Tow : Lexical Analysis

Input Buffering

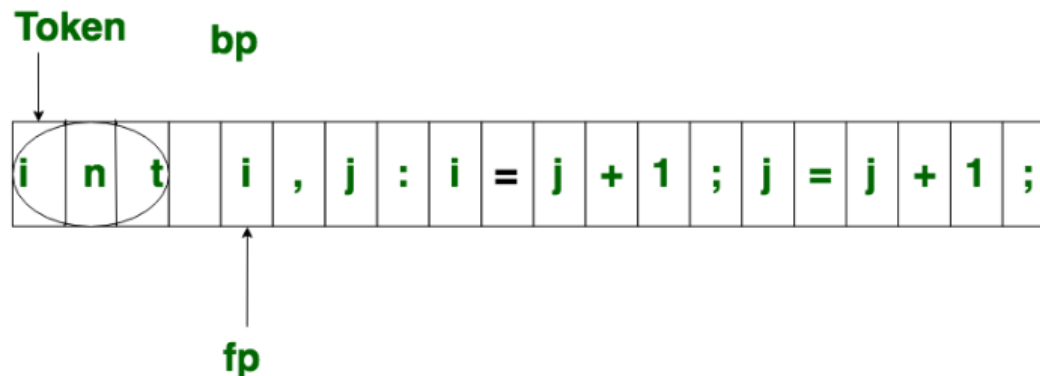
- The forward ptr moves ahead to search for end of lexeme.
- As soon as the blank space is encountered, it indicates end of lexeme. In above example as soon as ptr (fp) encounters a blank space the lexeme “int” is identified.



Chapter Tow : Lexical Analysis

Input Buffering

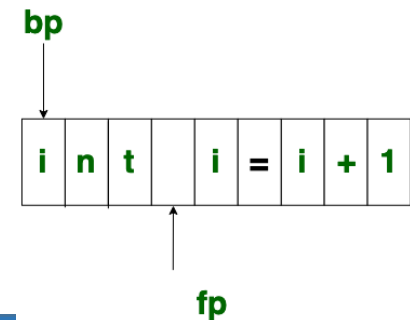
- The fp will be moved ahead at white space, when fp encounters white space, it ignore and moves ahead. Then both the begin ptr(bp) and forward ptr(fp) are set at next token.



Chapter Tow : Lexical Analysis

One Buffer Scheme:

- In this scheme, only one buffer is used to store the input string.
- The problem with this scheme is that if lexeme is very long then it crosses the buffer boundary, to scan rest of the lexeme the buffer has to be refilled, that makes overwriting the first of lexeme.



Chapter Tow : Lexical Analysis

Two Buffer Scheme:

- To overcome the problem of one buffer scheme, in this method two buffers are used to store the input string. the first buffer and second buffer are scanned alternately.
 - When end of current buffer is reached the other buffer is filled.
 - The only problem with this method is that if length of the lexeme is longer than length of the buffer then scanning input cannot be scanned completely.
-

Chapter Tow : Lexical Analysis

Two Buffer Scheme:

- Initially both the bp and fp are pointing to the first character of first buffer.
 - Then the fp moves towards right in search of end of lexeme. as soon as blank character is recognized, the string between bp and fp is identified as corresponding token.
 - To identify, the boundary of first buffer end of buffer character should be placed at the end first buffer.
-

Chapter Tow : Lexical Analysis

Two Buffer Scheme:

- Similarly end of second buffer is also recognized by the end of buffer mark present at the end of second buffer.
 - when fp encounters first eof, then one can recognize end of first buffer and hence filling up second buffer is started.
 - In the same way when second eof is obtained then it indicates of second buffer. alternatively both the buffers can be filled up until end of the input program and stream of tokens is identified.
 - This eof character introduced at the end is calling Sentinel which is used to identify the end of buffer.
-

Chapter Tow : Lexical Analysis

Two Buffer Scheme:

